~\AppData\Local\Packages\5319275A.WhatsAppDesktop_cv1g1gvanyjgm\LocalState\sessions\88928DF2345388693F4AF5DFC2BED30FE0C23423\transfers\2026-41\03_Exp3_U3.cpp

```
          // OOPS EXPERIMENT-3 (Unit 3)
  ============================================================================================================
            //NAME:-Aditya Sheetal Loharkar
            //Roll No:-20
            //DIV:-SOAI 4
            //SUBJECT:-OOPS
==============================================================================================================


//CODE:-


#include <iostream>
#include <string>
using namespace std;

// Hybrid Inheritance - Diamond Problem
class Person {
protected:
    string name;
    int age;

public:
    Person() {
        name = "Unknown";
        age = 0;
    }

    Person(string n, int a) {
        name = n;
        age = a;
    }

    void display() {
        cout << "Name: " << name << endl;
        cout << "Age: " << age << endl;
    }
};

// Virtual inheritance to solve diamond problem
class Student : virtual public Person {
protected:
    int rollNumber;
    string course;

public:
    Student() : Person() {
        rollNumber = 0;
        course = "Unknown";
    }

    Student(string n, int a, int roll, string c)
        : Person(n, a) {
        rollNumber = roll;
        course = c;
    }

    void displayStudent() {
        display();
        cout << "Roll Number: " << rollNumber << endl;
        cout << "Course: " << course << endl;
    }
};

class Employee : virtual public Person {
protected:
    int employeeId;
```

```
    float salary;

public:
    Employee() : Person() {
        employeeId = 0;
        salary = 0;
    }

    Employee(string n, int a, int id, float s)
        : Person(n, a) {
        employeeId = id;
        salary = s;
    }

    void displayEmployee() {
        display();
        cout << "Employee ID: " << employeeId << endl;
        cout << "Salary: " << salary << endl;
    }
};

// Multiple inheritance - Hybrid
class WorkingStudent : public Student, public Employee {
private:
    int workHours;

public:
    WorkingStudent() : Person(), Student(), Employee() {
        workHours = 0;
    }

    WorkingStudent(string n, int a, int roll, string c,
                   int id, float s, int hours)
        : Person(n, a), Student(n, a, roll, c), Employee(n, a, id, s) {
        workHours = hours;
    }

    void displayWorkingStudent() {
        // Calling display() from Person class - now unambiguous
        display();
        cout << "Roll Number: " << rollNumber << endl;
        cout << "Course: " << course << endl;
        cout << "Employee ID: " << employeeId << endl;
        cout << "Salary: " << salary << endl;
        cout << "Work Hours: " << workHours << endl;
    }
};

// Class for demonstrating ambiguity resolution
class Base1 {
public:
    void show() {
        cout << "Base1 Show" << endl;
    }

    void display() {
        cout << "Base1 Display" << endl;
    }
};

class Base2 {
public:
    void show() {
        cout << "Base2 Show" << endl;
    }

    void print() {
        cout << "Base2 Print" << endl;
    }
};
```

```cpp

class Derived : public Base1, public Base2 {
public:
    void show() {
        // Override show to resolve ambiguity
        cout << "Derived Show" << endl;
    }

    void resolveAmbiguity() {
        // Using scope resolution operator
        Base1::show();  // Call Base1's show
        Base2::show();  // Call Base2's show
        display();      // No ambiguity - only in Base1
        print();        // No ambiguity - only in Base2
    }
};

int main() {
    cout << "===== Hybrid Inheritance - Working Student (Diamond Problem) =====" << endl;
    WorkingStudent ws("Bob Johnson", 25, 202, "Computer Science", 5001, 45000, 20);
    ws.displayWorkingStudent();

    cout << "\n===== Ambiguity Resolution =====" << endl;
    Derived d;
    d.show();  // Calls Derived's show
    d.resolveAmbiguity();

    return 0;
}
```